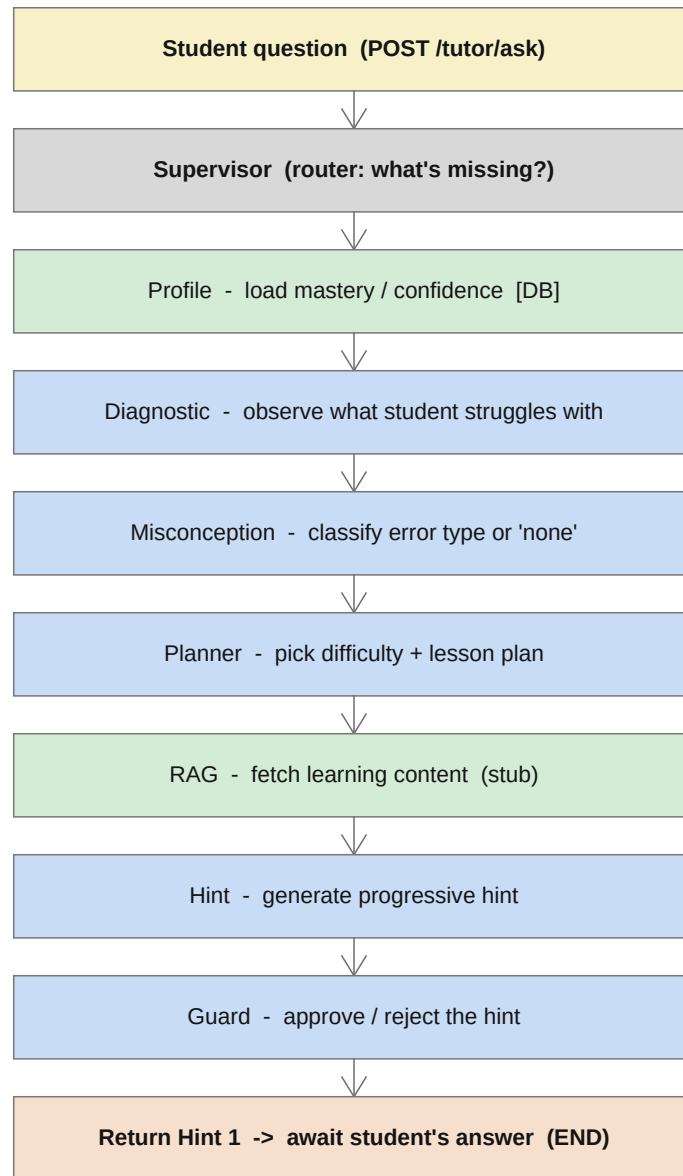


Adaptive AI Tutor

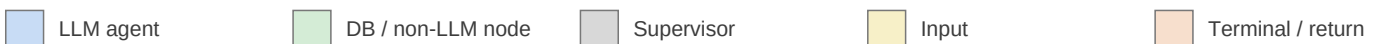
LangGraph multi-agent tutor - graph & flow explained

The tutor is a supervisor-routed state machine. A 'supervisor' re-runs after every agent and decides the next one purely from which state fields are filled (router.py). One /tutor/ask call runs the graph until it either delivers a hint (and pauses for the student's answer) or reaches a terminal branch (completed / escalated).

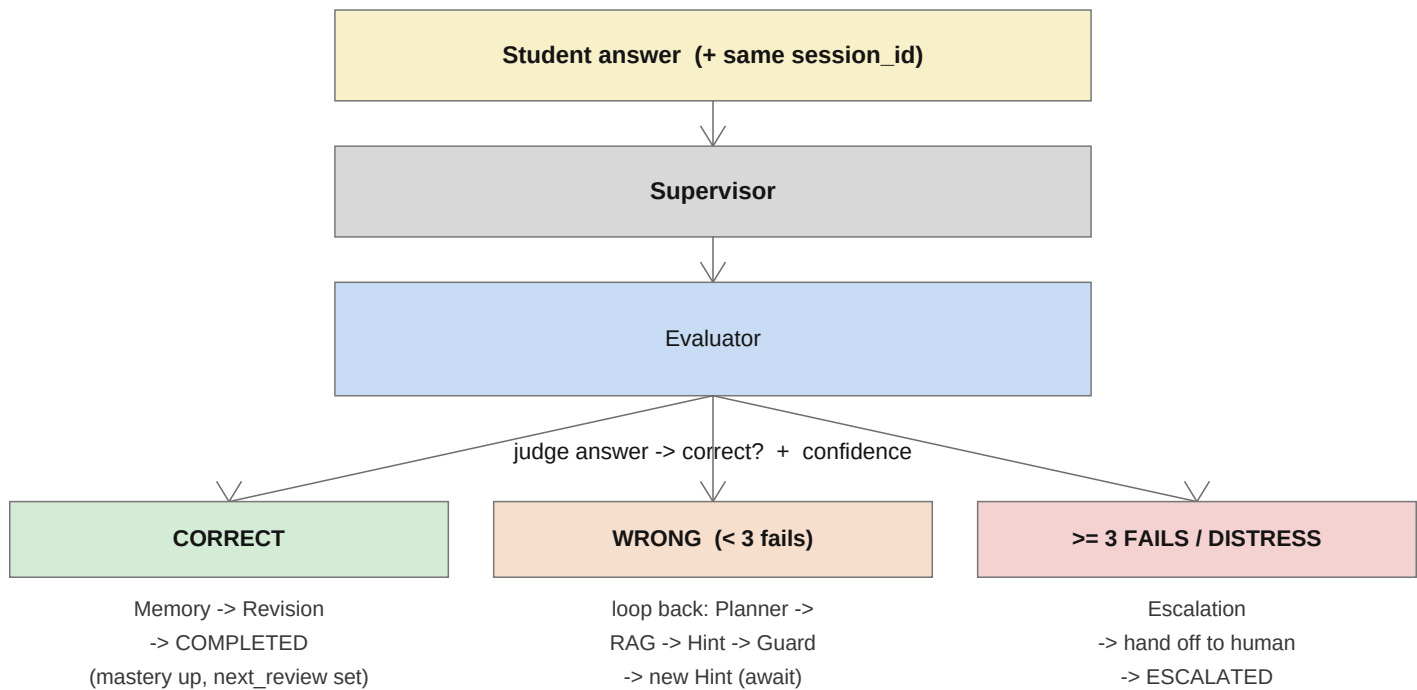
Flow A - First question -> Hint (Turn 1)



Every agent returns to the Supervisor, which re-routes to the next missing step. After Guard approves the hint, the turn ends (status stays 'active', awaiting your answer).



When you send the next message with the same session_id, the saved state is reloaded and 'awaiting' marks it as an answer. The Supervisor jumps straight to the Evaluator (it does NOT re-run profile/diagnostic/planner). The Evaluator judges the answer, updates the long-term profile (on EVERY answer), and writes an evidence row - then the router branches.



A wrong answer (fewer than 3 failures) increments the hint level and loops for another attempt - the session stays active. Three failures or detected distress escalates to a human. A correct answer finalizes the session (status = completed).

One file per agent under app/features/tutor/graph/nodes/. Each returns a partial state update.

Agent	Role	Input	Output
Supervisor	Route to the next agent	graph state	next node
Profile	Load student profile	student_id	mastery, confidence
Diagnostic	Observe the struggle (no teaching)	question	observations
Misconception	Classify the error	observations	label or 'none'
Planner	Difficulty + lesson plan	profile + misconception	plan
RAG	Fetch content (stub retriever)	concept	docs
Hint	One progressive hint (not the answer)	plan	hint text
Guard	Validate the hint	hint	APPROVE / REJECT
Evaluator	Judge answer, confidence; save profile+evidence	student answer	correct?, confidence
Memory	Finalize the success message	evaluation	completed message
Revision	Schedule next review (spaced rep.)	mastery	next_review date
Escalation	Hand off to a human teacher	failures / distress	escalation record

Routing rules (router.py)

The Supervisor returns the FIRST matching branch:

- 1. profile missing -> Profile
- 2. diagnostic missing -> Diagnostic
- 3. misconception missing -> Misconception
- 4. tutor_plan missing -> Planner
- 5. docs missing -> RAG
- 6. hint missing -> Hint (then Guard)
- 7. evaluation missing -> Evaluator (only if the message is an answer; else END/await)
- 8. wrong & failures < 3 -> Planner (loop, harder hint)
- 9. failures >= 3 / distress-> Escalation
- 10. correct -> Memory -> Revision -> END

One /tutor/ask call writes to several tables immediately (nothing waits for the end):

Table	When	What
conversation_history	every turn	2 rows: the user message + the assistant reply
tutor_sessions	every turn	1 row (created turn 1), state JSON + status overwritten
evidence_events	on an answer	1 row: correct?, error_type (Evaluator)
student_profile	on an answer	mastery, confidence, evidence_count, misconceptions updated
teacher_escalations	on escalation	1 row: reason (>=3 fails / distress)

Sessions vs conversation

tutor_sessions = the graph's working memory: ONE row per session, the 'state' JSON is overwritten each turn so the next turn can resume (current hint, failures, mastery...).

conversation_history = the transcript: MANY rows (2 per turn - user + assistant), append-only, linked to the session by session_id. One session -> many conversation rows.

Providers & observability

The model behind every agent is chosen by env (core/llm.py): Anthropic subscription or API key, OpenAI, or Gemini - a config change, not a code change. With a LANGSMITH_API_KEY set, each agent call is traced as 'agent:<stage>' (prompt in, output out). With DEBUG_AGENT_IO=true the same {agent: {input, output}} dict is printed and returned in the response.